

USART2 (VCP) 通信改善案

対象ボード: NUCLEO-F446RE

通信経路: STM32 USART2 (PA2/PA3) → ST-Link VCP → MicroB USB → PC (ROS)

作成日: 2026-02-14

現状の構成

項目	設定値
ペリフェラル	USART2 (PA2: TX, PA3: RX)
ボーレート	115200 bps
送信方式	HAL_UART_Transmit (ブロッキング/ポーリング)
受信方式	HAL_UART_Receive_IT (1バイト割り込み)
DMA	未設定
送信データ	int32_t × 3 (エンコード値) + COBS/ヘッダ ≒ 約20バイト
送信周期	10ms (100Hz)

問題点

1. 送信がブロッキング（優先度：高）

```
// UartLink.cpp:19
HAL_UART_Transmit(huart_ptr, data, size, 50); // タイムアウト50ms
```

- 送信完了まで CPU が待機し続ける
- 約20バイト / 115200bps ≒ **約1.7ms のCPUブロック**が10ms毎に発生
- CPU時間の約17%が UART 送信待ちに浪費される
- 将来的にモーター制御等の処理を追加した場合、制御周期に影響する可能性あり

2. 受信が1バイト割り込み（優先度：中）

```
// UartLink.cpp:13
HAL_UART_Receive_IT(huart_ptr, &receive_buffer[0], 1);
```

- 1バイト受信するたびに割り込み → HALコールバック → 処理のオーバーヘッドが発生
- ROS 側からの送信が増えると割り込み頻度が上がり、他の処理に影響する

3. DMA 未使用（優先度：中）

- `.ioc` で USART2 に DMA が設定されていない
- 送受信ともにCPU介在が必要な状態

4. ボーレートの選択（優先度：低）

- 115200bps は安全な値だが、VCP 経由では ST-Link が USB Full Speed (12Mbps) で吸収するため、STM32 側のボーレートを上げて問題ない可能性がある
- ただし ST-Link のファームウェアバージョンによっては高ボーレートで不安定になるケースもあるため、変更する場合は要テスト

改善案

改善1: 送信の DMA 化（効果：大、難易度：低～中）

変更内容:

1. CubeMX で USART2_TX に DMA チャンネルを割り当て
2. `HAL_UART_Transmit` → `HAL_UART_Transmit_DMA` に変更
3. 送信完了コールバック `HAL_UART_TxCpltCallback` を追加

効果:

- 送信中の CPU ブロック（約1.7ms/回）がほぼゼロに
- CPU を他の処理に使える

注意点:

- DMA 送信中はバッファ内容を変更してはいけないため、ダブルバッファリングまたは送信完了フラグの管理が必要
- `UartLink.cpp` の `send_raw()` を改修する必要あり

改善2: 受信の DMA + IDLE Line 検出化（効果：中、難易度：中）

変更内容:

1. CubeMX で USART2_RX に DMA チャンネルを割り当て（Circular モード）
2. IDLE Line 割り込みを有効化
3. IDLE 検出時にフレーム境界を判定して `on_receive_raw()` を呼ぶ

効果:

- 1バイトごとの割り込みが不要になる
- フレーム単位でまとめて処理できる

注意点:

- `UartLink` クラスの受信ロジックを大幅に書き換える必要あり
- COBS のデリミタ (0x00) との整合性を考慮する必要がある

改善3: ボーレートの引き上げ（効果：小、難易度：低）

変更内容:

- CubeMX で USART2 のボーレートを 921600 等に変更
- ROS 側のシリアルポート設定も合わせる

効果:

- ブロッキング送信のままでも待ち時間を約1/8に短縮可能 (1.7ms → 約0.2ms)

注意点:

- VCP 経由の場合、多くの ST-Link ファームウェアでボーレート設定は無視されるが、一部バージョンでは尊重される → 要テスト

推奨の優先順位

順位	改善案	理由
1	送信の DMA 化	最も効果が大きく、CPU 負荷を直接改善
2	ボーレート引き上げ	DMA が難しい場合の簡易代替策
3	受信の DMA + IDLE 化	受信量が増えた場合に検討

現状の用途（エンコード値3個の送信のみ）であれば、データ量が少ないため大きな問題にはならない。

ただし、将来的にサブスクライバーの追加やモーター制御を組み込む場合は、改善1（送信DMA化）を推奨する。

具体的なコード変更案

以下に、各改善案を実装する場合の CubeMX 設定手順と具体的なコード変更例を示す。

改善1: 送信の DMA 化 — 実装ガイド

Step 1: CubeMX 設定

1. `F3RC_TEAMB_HAL.ioc` を CubeMX で開く
2. **Connectivity** → **USART2** → **DMA Settings** タブを開く
3. **Add** をクリックし、以下を設定：
 - DMA Request: `USART2_TX`
 - Direction: `Memory to Peripheral`
 - Priority: `Low` (他にDMAを使っていないため)
 - Mode: `Normal` (送信は毎回手動トリガのため Circular 不要)
 - Data Width: `Byte / Byte`
4. **NVIC Settings** タブで `DMA Stream IRQ` が有効であることを確認
5. コード生成を実行

Step 2: UartLink.hpp の変更

```
#pragma once

#include "CommunicationBase.hpp"
#include "main.h"

class UartLink : public CobsEncodedCommunicationBase {
public:
    UartLink(UART_HandleTypeDef* huart, uint8_t device_id=0);

    void start();

    // 送信処理
    void send_raw(uint8_t* data, uint8_t size) override;

    // 受信割り込み時に呼ぶ
    void interrupt();

    // ★追加: 送信完了コールバック
    void on_tx_complete();

    // ★追加: 送信中かどうか
    bool is_tx_busy() const { return tx_busy; }

private:
    uint8_t receive_buffer[BUFFER_SIZE];
    uint8_t receive_buffer_index = 0;

    // ★追加: DMA送信用バッファ (送信中に書き換えられないようにコピーを保持)
    uint8_t tx_dma_buffer[BUFFER_SIZE];
    volatile bool tx_busy = false;

    UART_HandleTypeDef* huart_ptr;
};
```

Step 3: UartLink.cpp の変更

```
#include "UartLink.hpp"
#include <string.h>

UartLink::UartLink(UART_HandleTypeDef* huart, uint8_t device_id)
    : CobsEncodedCommunicationBase(device_id)
{
    huart_ptr = huart;
}

void UartLink::start()
{
    receive_buffer_index = 0;
```

```

    HAL_UART_Receive_IT(huart_ptr, &receive_buffer[0], 1);
}

// ★変更: DMA送信に切り替え
void UartLink::send_raw(uint8_t* data, uint8_t size)
{
    // 前回の送信がまだ完了していない場合は待つ (最低限の安全策)
    uint32_t timeout = HAL_GetTick();
    while (tx_busy) {
        if (HAL_GetTick() - timeout > 50) {
            // タイムアウト: 強制的にリセット
            tx_busy = false;
            break;
        }
    }

    // 送信データを DMA バッファにコピー
    memcpy(tx_dma_buffer, data, size);
    tx_busy = true;
    HAL_UART_Transmit_DMA(huart_ptr, tx_dma_buffer, size);
}

// ★追加: 送信完了コールバック
void UartLink::on_tx_complete()
{
    tx_busy = false;
}

// 受信割り込み (変更なし)
void UartLink::interrupt()
{
    uint8_t c = receive_buffer[receive_buffer_index];
    receive_buffer_index++;

    if (receive_buffer_index >= BUFFER_SIZE - 6) {
        receive_buffer_index = 0;
        HAL_UART_Receive_IT(huart_ptr, &receive_buffer[0], 1);
        return;
    }

    if (c == 0x00) {
        HAL_UART_Receive_IT(huart_ptr, &receive_buffer[0], 1);
        uint8_t copied[BUFFER_SIZE];
        memcpy(copied, receive_buffer, receive_buffer_index);
        on_receive_raw(copied, receive_buffer_index);
        receive_buffer_index = 0;
    } else {
        HAL_UART_Receive_IT(huart_ptr, &receive_buffer[receive_buffer_index], 1);
    }
}

```

Step 4: `main.cpp` に送信完了コールバックを追加

```
// ★追加: 送信完了割り込み
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART2) {
        uart_link.on_tx_complete();
    }
}
```

[!IMPORTANT] `main.cpp` の既存コールバック (`HAL_UART_RxCpltCallback`, `HAL_UART_ErrorCallback`) の近くに配置するとよい。
`loop()` 内の `encoder_pub.publish(...)` はそのまま変更不要。

改善2: 受信の DMA + IDLE Line 検出 — 実装ガイド

[!WARNING] この改善は `UartLink` クラスの受信ロジックを大きく変更するため、改善1を先に実施・検証した後に取り組むことを推奨。

Step 1: CubeMX 設定

1. **Connectivity** → **USART2** → **DMA Settings** で **USART2_RX** を追加 :

- Direction: **Peripheral to Memory**
- Mode: **Circular** (受信は連続的に行うため)
- Data Width: **Byte / Byte**

2. コード生成を実行

Step 2: `UartLink.hpp` の変更

```
class UartLink : public CobsEncodedCommunicationBase {
public:
    UartLink(UART_HandleTypeDef* huart, uint8_t device_id=0);

    void start();
    void send_raw(uint8_t* data, uint8_t size) override;

    // ★変更: IDLE検出時に呼ぶ (旧interruptの代わり)
    void on_idle_detected();

    void on_tx_complete();
    bool is_tx_busy() const { return tx_busy; }

private:
    // ★変更: DMA用の大きめリングバッファ
    static const uint16_t RX_DMA_BUF_SIZE = 256;
    uint8_t rx_dma_buffer[RX_DMA_BUF_SIZE];
    uint16_t rx_read_pos = 0; // 前回読み取った位置

    // フレーム組み立て用
    uint8_t frame_buffer[BUFFER_SIZE];
    uint8_t frame_index = 0;
```

```

uint8_t tx_dma_buffer[BUFFER_SIZE];
volatile bool tx_busy = false;

UART_HandleTypeDef* huart_ptr;
};

```

Step 3: UartLink.cpp の変更（受信部分）

```

void UartLink::start()
{
    rx_read_pos = 0;
    frame_index = 0;
    // DMA Circular モードで受信開始
    HAL_UARTEx_ReceiveToIdle_DMA(huart_ptr, rx_dma_buffer, RX_DMA_BUF_SIZE);
    // Half Transfer 割り込みを無効化（不要なため）
    __HAL_DMA_DISABLE_IT(huart_ptr->hdmarx, DMA_IT_HT);
}

// IDLE 検出時 または DMA Transfer Complete 時に呼ばれる
void UartLink::on_idle_detected()
{
    // DMAが現在書き込んでいる位置を取得
    uint16_t write_pos = RX_DMA_BUF_SIZE
        - __HAL_DMA_GET_COUNTER(huart_ptr->hdmarx);

    // リングバッファから新しいデータを読み出し
    while (rx_read_pos != write_pos) {
        uint8_t c = rx_dma_buffer[rx_read_pos];
        rx_read_pos = (rx_read_pos + 1) % RX_DMA_BUF_SIZE;

        if (c == 0x00) {
            // COBS デリミタを検出 → フレーム完成
            if (frame_index > 0) {
                on_receive_raw(frame_buffer, frame_index);
            }
            frame_index = 0;
        } else {
            if (frame_index < BUFFER_SIZE - 6) {
                frame_buffer[frame_index++] = c;
            } else {
                // オーバーフロー: フレーム破棄
                frame_index = 0;
            }
        }
    }
}

```

Step 4: main.cpp のコールバック変更

```
// ★変更: RxCpltCallback の代わりに RxEventCallback を使用
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size) {
    if (huart->Instance == USART2) {
        uart_link.on_idle_detected();
    }
}

// ★削除可能: 旧 HAL_UART_RxCpltCallback は不要になる
// void HAL_UART_RxCpltCallback(...) { ... }
```

[!NOTE] `HAL_UARTEx_RxEventCallback` は HAL の UART 拡張 API で、IDLE 検出時と DMA Transfer Complete 時の両方で呼ばれる。

`HAL_UARTEx_ReceiveToIdle_DMA` と組み合わせて使う。

改善3: ボーレート引き上げ — 実装ガイド

Step 1: CubeMX 設定

1. **Connectivity** → **USART2** → **Parameter Settings** を開く
2. **Baud Rate** を `921600` (または `460800`) に変更
3. コード生成を実行
4. 生成された `usart.c` で以下の行が変わっていることを確認:

```
- huart2.Init.BaudRate = 115200;
+ huart2.Init.BaudRate = 921600;
```

Step 2: ROS 側の設定変更

ROS の serial ノード設定でボーレートを合わせる:

```
# ROS2 の場合 (例: launch ファイル内)
parameters:
  - serial_port: "/dev/ttyACM0"
  - baud_rate: 921600
```

```
# pyserial の場合
ser = serial.Serial('/dev/ttyACM0', 921600)
```

注意事項

[!CAUTION] VCP 経由の場合、ST-Link V2/V2-1 のファームウェアによっては **ホスト側のボーレート設定を STM32 側に反映する** 動作をするものがあり、STM32 側と PC 側のボーレートが一致していない

と通信できないケースがある。
変更後は必ず実機で通信テストを行うこと。

改善の組み合わせパターン

パターン	変更範囲	効果	おすすめ度
A: 改善1のみ	UartLink + CubeMX DMA追加	送信CPUブロック解消	★★★
B: 改善3のみ	CubeMX + ROS設定のみ	送信時間を1/8に短縮	★★
C: 改善1 + 改善2	UartLink 全面改修 + CubeMX	送受信ともに最適化	★★★★
D: 改善1 + 改善2 + 改善3	フル改修	最大効率	★★★★★

推奨: まずは **パターンA（送信DMA化のみ）** を実施し、動作確認後に必要に応じて B → C の順で拡張していく。